

WS2.4 – Weerbaarheid software

Future Factory Applicatie



Groep: 10 – Firewall Force

Groepsleden: Bavli, Jonas, Lucas, Vo

Inleiding

Voor Workshop 4 bouwen wij verder op de casus **Future Factory**, dezelfde Laravel-applicatie die wij ook in Workshop 3 hebben gebruikt. Future Factory is geschikt als analysecasus, omdat de applicatie meerdere gebruikersrollen, formulieren, CRUD-functionaliteit, database-interactie en gevoelige configuratie bevat. Daardoor kunnen wij de weerbaarheid van zowel software als onderliggende componenten goed bespreken.

Workshop 4 richt zich op belangrijke elementen die bijdragen aan de weerbaarheid van digitale systemen, zodat ontwikkelteams beter begrijpen wat nodig is om veilige en weerbare producten te realiseren. In de workshop komen onder meer statische analyse, dynamische tests, open source analyse, secure-by-design, algemene kwetsbaarheden, hardening en hardware security terug.

OPDRACHT 1

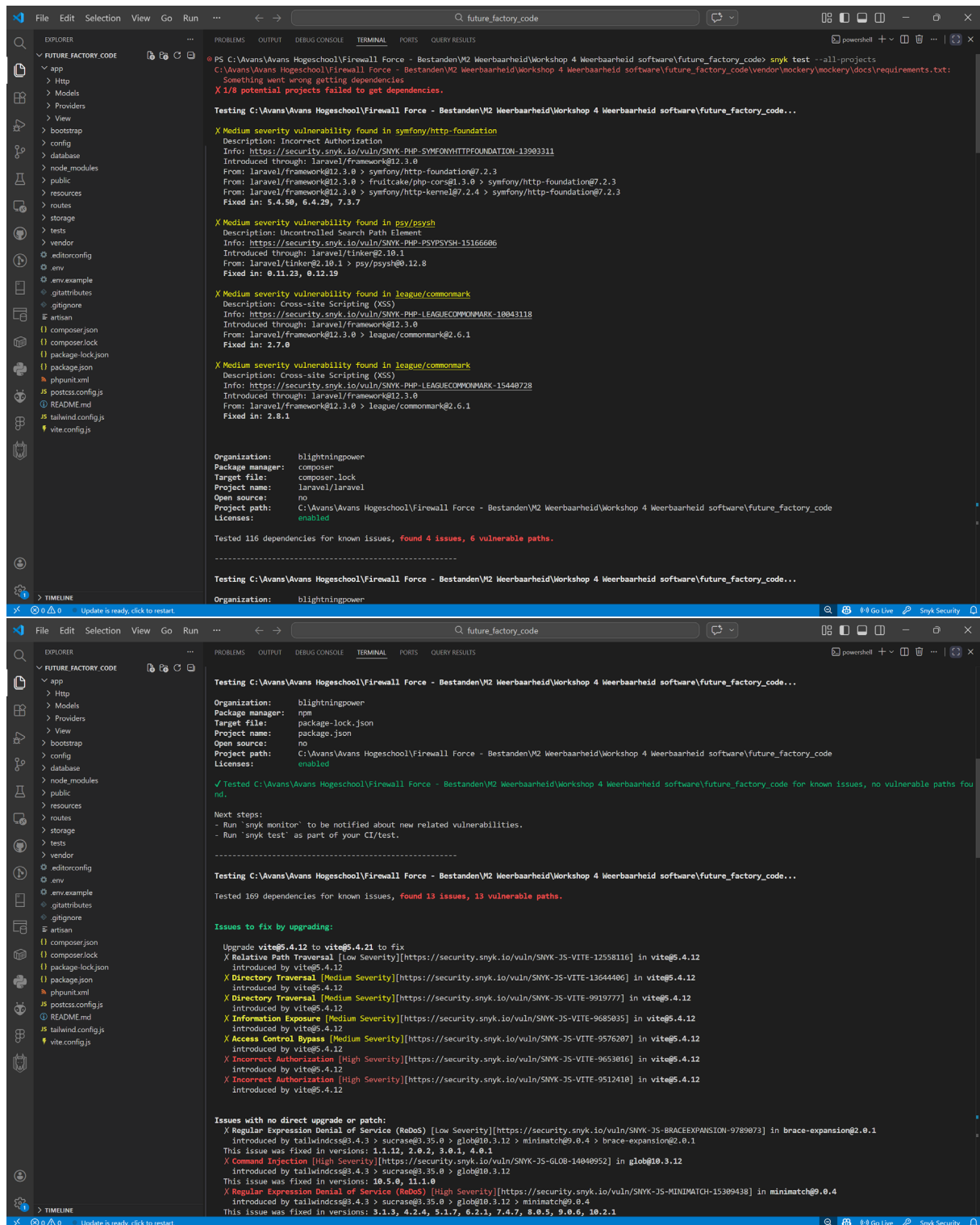
1. SAST en open source analyse op een serieuze applicatie

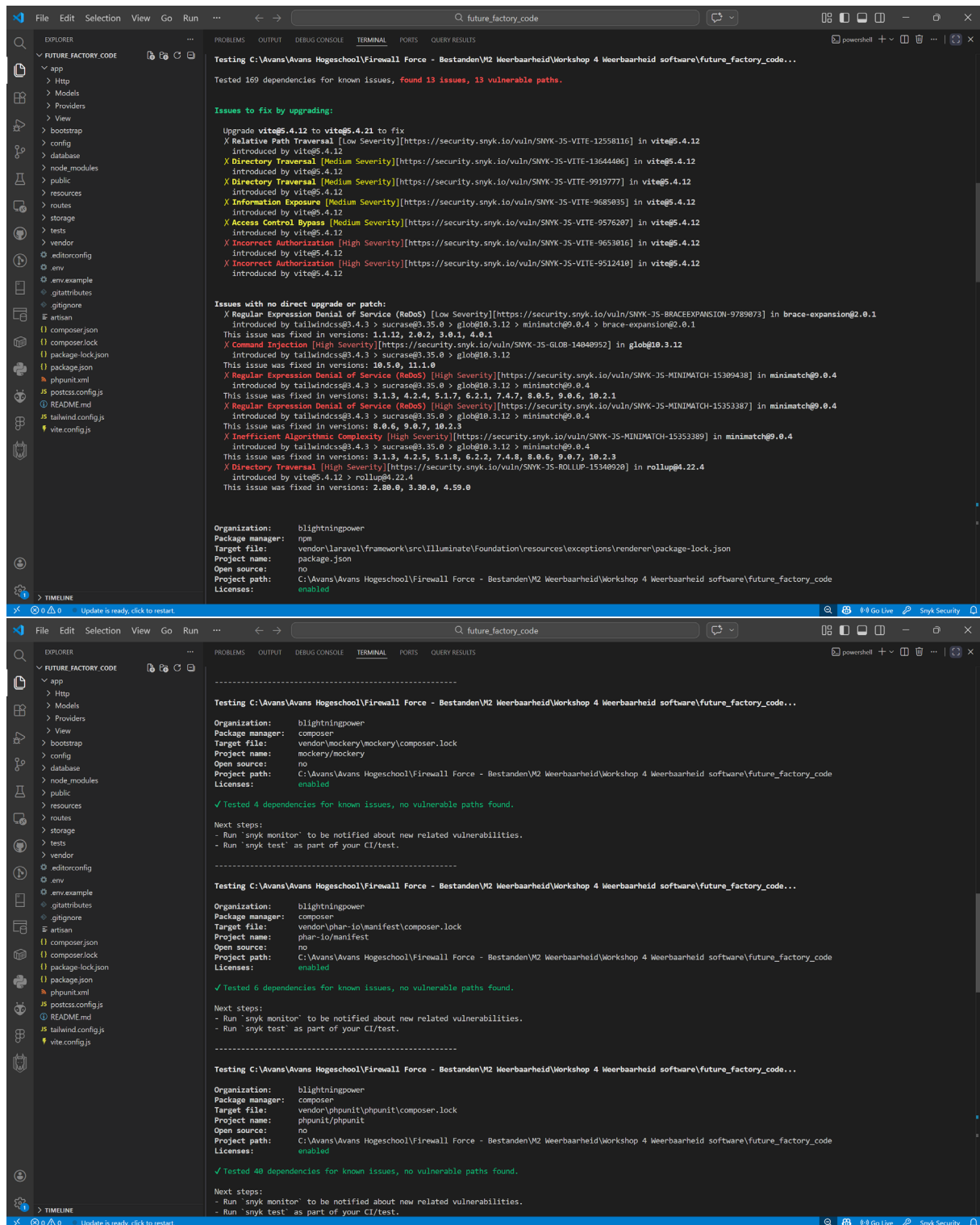
Voor deze analyse is de applicatie Future Factory onderworpen aan een technische security-scan. Deze Laravel-applicatie bevat kritieke componenten zoals gebruikersauthenticatie, database-interacties en configuratiebeheer, wat het een relevant doelwit maakt voor een Secure Software Development Lifecycle (SSDLC) evaluatie.

Toelichting op de aanpak

De uitgevoerde analyse bestaat uit twee technische methodieken die centraal staan in de verificatiefase van softwareontwikkeling:

- **Static Application Security Testing (SAST):** Het scannen van de eigen broncode op programmeerfouten en onveilige patronen, zoals hardcoded geheimen.
- **Open Source Analysis (SCA):** Het controleren van externe bibliotheken en afhankelijkheden op bekende kwetsbaarheden (CVE's).





```
File Edit Selection View Go Run ...
future_factory_code

EXPLORER
FUTURE_FACTORY_CODE
  app
  Http
  Models
  Providers
  View
  bootstrap
  config
  database
  node_modules
  public
  resources
  routes
  storage
  tests
  vendor
  editorconfig
  .env
  .env.example
  gitattributes
  gitignore
  artisan
  composer.json
  composer.lock
  package-lock.json
  package.json
  phpunit.xml
  postcss.config.js
  README.md
  tailwind.config.js
  vite.config.js

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
Testing C:\Avans\Avans Hogeschool\Firewall Force - Bestanden\V2 Weerbaarheid\Workshop 4 Weerbaarheid software\future_factory_code...

Organization: blightningpower
Package manager: composer
Target file: vendor\theseer\tokenizer\composer.lock
Project name: theseer/tokenizer
Open source: no
Project path: C:\Avans\Avans Hogeschool\Firewall Force - Bestanden\V2 Weerbaarheid\Workshop 4 Weerbaarheid software\future_factory_code
Licenses: enabled

✓ Tested 4 dependencies for known issues, no vulnerable paths found.

Next steps:
- Run 'snyk monitor' to be notified about new related vulnerabilities.
- Run 'snyk test' as part of your CI/test.

Tested 7 projects, 2 contained vulnerable paths.

PS C:\Avans\Avans Hogeschool\Firewall Force - Bestanden\V2 Weerbaarheid\Workshop 4 Weerbaarheid software\future_factory_code> snyk code test

Testing C:\Avans\Avans Hogeschool\Firewall Force - Bestanden\V2 Weerbaarheid\Workshop 4 Weerbaarheid software\future_factory_code ...

Open Issues

X [LOW] Use of Password Hash With Insufficient Computational Effort
Path: tests/Feature/Auth/EmailVerificationTest.php, line 24
Info: SHA1 hash (used in sha1) is insecure. Consider changing it to a secure hashing algorithm.

X [LOW] Use of Password Hash With Insufficient Computational Effort
Path: tests/Feature/Auth/EmailVerificationTest.php, line 40
Info: SHA1 hash (used in sha1) is insecure. Consider changing it to a secure hashing algorithm.

X [MEDIUM] Use of Hardcoded Passwords
Path: app/Models/User.php, line 47
Info: Do not hardcode passwords in code. Found a hardcoded password used in variable.

Test Summary
Organization: blightningpower
Test type: Static code analysis
Project path: C:\Avans\Avans Hogeschool\Firewall Force - Bestanden\V2 Weerbaarheid\Workshop 4 Weerbaarheid software\future_factory_code

Total issues: 3
Ignored issues: 0 [ 0 HIGH 0 MEDIUM 0 LOW ]
Open issues: 3 [ 0 HIGH 1 MEDIUM 2 LOW ]

Tip
Update is ready, click to restart.
```

```
File Edit Selection View Go Run ...
future_factory_code

EXPLORER
FUTURE_FACTORY_CODE
  app
  Http
  Models
  Providers
  View
  bootstrap
  config
  database
  node_modules
  public
  resources
  routes
  storage
  tests
  vendor
  editorconfig
  .env
  .env.example
  gitattributes
  gitignore
  artisan
  composer.json
  composer.lock
  package-lock.json
  package.json
  phpunit.xml
  postcss.config.js
  README.md
  tailwind.config.js
  vite.config.js

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
Testing C:\Avans\Avans Hogeschool\Firewall Force - Bestanden\V2 Weerbaarheid\Workshop 4 Weerbaarheid software\future_factory_code ...

Open Issues

X [LOW] Use of Password Hash With Insufficient Computational Effort
Path: tests/Feature/Auth/EmailVerificationTest.php, line 24
Info: SHA1 hash (used in sha1) is insecure. Consider changing it to a secure hashing algorithm.

X [LOW] Use of Password Hash With Insufficient Computational Effort
Path: tests/Feature/Auth/EmailVerificationTest.php, line 40
Info: SHA1 hash (used in sha1) is insecure. Consider changing it to a secure hashing algorithm.

X [MEDIUM] Use of Hardcoded Passwords
Path: app/Models/User.php, line 47
Info: Do not hardcode passwords in code. Found a hardcoded password used in variable.

Test Summary
Organization: blightningpower
Test type: Static code analysis
Project path: C:\Avans\Avans Hogeschool\Firewall Force - Bestanden\V2 Weerbaarheid\Workshop 4 Weerbaarheid software\future_factory_code

Total issues: 3
Ignored issues: 0 [ 0 HIGH 0 MEDIUM 0 LOW ]
Open issues: 3 [ 0 HIGH 1 MEDIUM 2 LOW ]

Tip
To view ignored issues, use the --include-ignores option.

Tip
Some capabilities, including the ability to apply ignores, are unavailable. Retest the project with the --remote-repo-url parameter or from within a repository to enable full functionality.

PS C:\Avans\Avans Hogeschool\Firewall Force - Bestanden\V2 Weerbaarheid\Workshop 4 Weerbaarheid software\future_factory_code> snyk monitor

Monitoring C:\Avans\Avans Hogeschool\Firewall Force - Bestanden\V2 Weerbaarheid\Workshop 4 Weerbaarheid software\future_factory_code (package.json)...

Explore this snapshot at https://app.snyk.io/org/blightningpower/project/1da08288-56a0-481d-b4c6-430cda46b69/history/f24808c0-8dbf-4bef-8590-cealacd83c3e

Tip: Detected multiple supported manifests (5), use --all-projects to scan all of them at once.

Notifications about newly disclosed issues related to these dependencies will be emailed to you.

PS C:\Avans\Avans Hogeschool\Firewall Force - Bestanden\V2 Weerbaarheid\Workshop 4 Weerbaarheid software\future_factory_code>

Tip
Update is ready, click to restart.
```

Belangrijkste Bevindingen

De onderstaande tabel bevat de vijf meest kritieke resultaten uit de Snyk-scan, inclusief het bijbehorende hersteladvies.

Nr	Type	Ernst	Bevinding	Advies
1	Open Source	High	Command Injection in glob@10.3.12	Update glob naar versie 10.5.0 of 11.1.0 om willekeurige code-executie te voorkomen.
2	Open Source	High	Incorrect Authorization in vite@5.4.12	Update vite naar versie 5.4.21. Dit voorkomt dat aanvallers toegangscontroles omzeilen.
3	Open Source	High	Directory Traversal in rollup@4.22.4	Update rollup naar versie 4.59.0. Dit voorkomt dat aanvallers buiten de bedoelde mappen kunnen lezen.
4	SAST	Medium	Hardcoded Passwords in app/Models/User.php	Verwijder wachtwoorden uit de broncode. Gebruik omgevingsvariabelen (.env) of veilige 'secrets management'.
5	Open Source	Medium	Cross-site Scripting (XSS) in league/commonmark	Update naar versie 2.8.1 om te voorkomen dat kwaadaardige scripts in de browser van gebruikers worden uitgevoerd.

Conclusie SAST/open source analyse

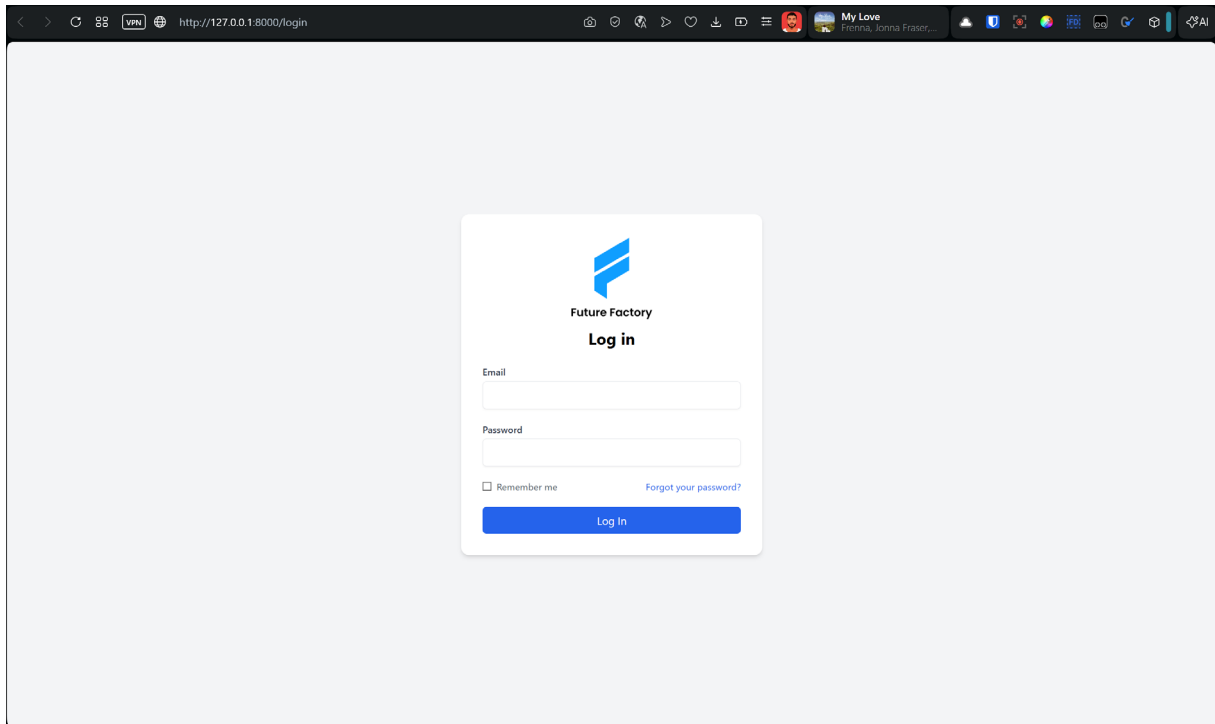
De technische scan bevestigt dat de weerbaarheid van Future Factory momenteel wordt beperkt door verouderde afhankelijkheden en onveilige opslag van inloggegevens. Hoewel de architectuur een bruikbare basis biedt, is het essentieel om een Software Bill of Materials (SBOM) bij te houden om inzicht te houden in het aanvalsoppervlak van gebruikte componenten. Door de bovenstaande updates door te voeren, wordt de ketenbeveiliging aanzienlijk versterkt.

2. DAST met ZAP

Toelichting op de aanpak

In de verificatiefase van de SSDLC is een Dynamic Application Security Testing (DAST) uitgevoerd. Dit is een black-box aanpak waarbij de draaiende applicatie op een stabiele testomgeving (<http://127.0.0.1:8000>) wordt geanalyseerd zonder kennis van de broncode. Het doel is om run-time bugs en configuratiefouten te ontdekken die door aanvallers, zoals script kiddies, misbruikt kunnen worden. Voor deze test is de tool OWASP ZAP gebruikt.

De scan is uitgevoerd op de lokale testomgeving: <http://127.0.0.1:8000/login>.



[illegible]

Bevindingen van de scan

De geautomatiseerde scan resulteerde in 3 Medium en 5 Low alerts. De vijf meest relevante bevindingen voor de weerbaarheid van Future Factory zijn:

Nr	Bevinding	Ernst	Omschrijving	Advies
1	Absence of Anti-CSRF Tokens	Medium	ZAP detecteerde formulieren (zoals bij /forgot-password) die mogelijk niet beschermd zijn tegen Cross-Site Request Forgery.	Controleer of de @csrf directive in alle Laravel Blade-formulieren aanwezig is.
2	Anti-clickjacking Header Missing	Medium	De X-Frame-Options header ontbreekt, waardoor de site in een frame geladen kan worden voor UI-redressing aanvallen.	Voeg de header X-Frame-Options: SAMEORIGIN toe aan de globale middleware in Laravel.
3	Cookie No HttpOnly Flag	Medium	Sessiecookies kunnen via scripts worden uitgelezen, wat het risico op sessie-overname vergroot.	Configureer config/session.php zodat http_only standaard op true staat.
4	X-Content-Type-Options Header Missing	Low	De browser kan het content-type "raden" (sniffing), wat onbedoelde script-executie mogelijk maakt.	Voeg de header X-Content-Type-Options: nosniff toe aan alle server-responses.
5	Application Error Disclosure	Low	De applicatie lekt mogelijk technische details in foutmeldingen bij niet-bestaande URL's.	Zorg dat APP_DEBUG=false strikt is ingesteld in de productie-omgeving.

Conclusie DAST

De ZAP-test bewijst dat de applicatie weliswaar functioneel is, maar op runtime-niveau nog kwetsbaarheden bevat rondom sessiebeheer en browser-interactie. Het implementeren van deze ontbrekende beveiligingsheaders is essentieel om de weerbaarheid tegen geautomatiseerde aanvallen te verhogen.

3. Secure-by-design en secure-by-default requirements

Voor de cyberweerbaarheid van Future Factory zijn de volgende eisen opgesteld. Weerbaarheid wordt hierbij gedefinieerd als het vermogen om aanvallen te anticiperen, te weerstaan, ervan te herstellen en je aan te passen.

Secure-by-design requirements voor Future Factory

Input Validatie: Alle gebruikersinvoer moet aan de serverzijde worden gevalideerd op type, lengte en formaat om injecties te voorkomen.

- **Referentie: MITRE CWE-20 (Improper Input Validation) en OWASP Top Ten.**

Toegangscontrole (Least Privilege): Gebruikers mogen standaard geen rechten hebben; autorisatie moet expliciet per rol en actie worden afgedwongen in de controllers.

- **Referentie: OWASP A01:2021 (Broken Access Control) en MITRE ATT&CK.**

Cryptografische Weerbaarheid: Wachtwoorden mogen uitsluitend worden opgeslagen met moderne algoritmen zoals Bcrypt; verouderde SHA1-hashes moeten worden verwijderd.

- **Referentie: MITRE CWE-327 (Use of a Broken Cryptographic Algorithm).**

Secure-by-default requirements voor Future Factory

Hardened Headers & Cookies: Alle HTTP-responses moeten standaard voorzien zijn van beveiligingsheaders (X-Frame-Options, X-Content-Type-Options) en cookies moeten de HttpOnly en Secure vlaggen dragen.

- **Referentie: MITRE ATT&CK T1539 (Steal Web Session Cookie).**

Minimale Attack Surface: Debugging-functionaliteiten en uitgebreide foutmeldingen staan standaard uitgeschakeld in elke omgeving die niet lokaal is.

- **Referentie: MITRE CWE-209 (Information Exposure Through Error Message).**

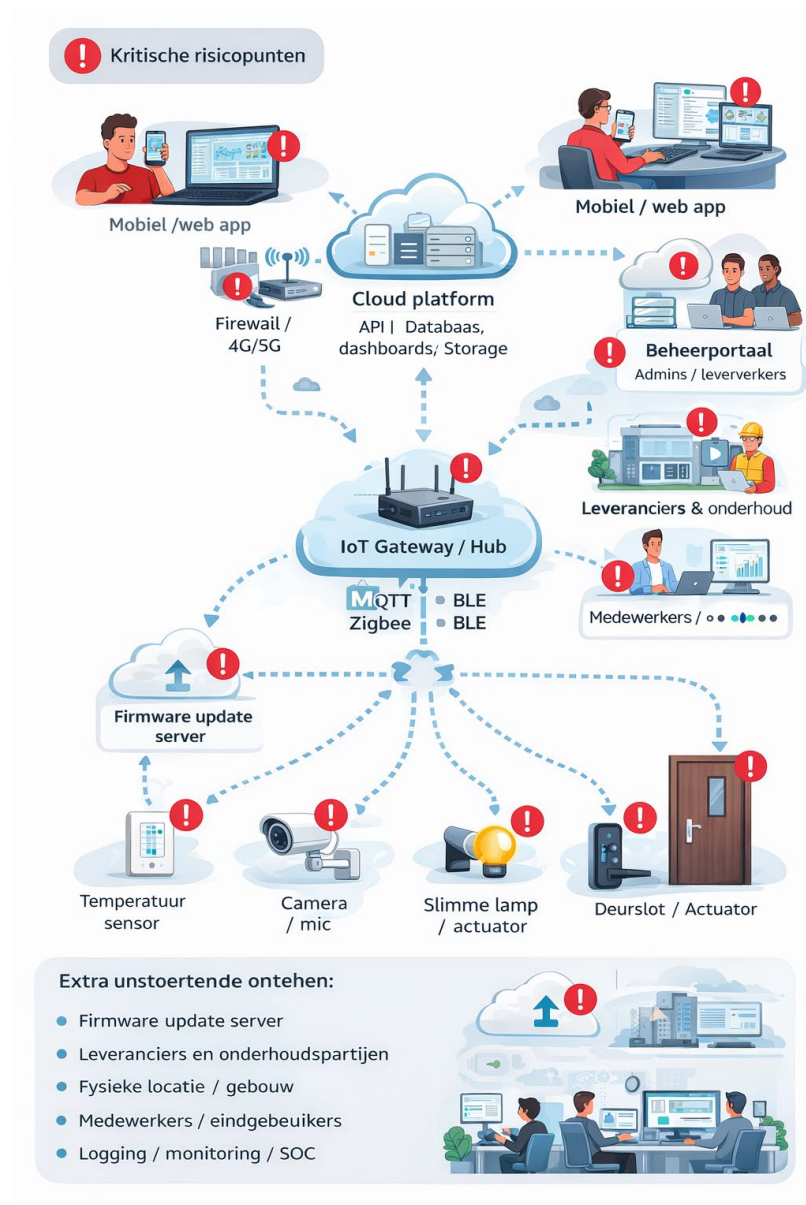
Dependency Management: De applicatie mag alleen gebouwd worden met dependencies die via een geautomatiseerde scan (SCA) zijn goedgekeurd en opgenomen in de SBOM.

- **Referentie: CRA (Cyber Resilience Act).**

4. Werkende voorbeelden van vijf algemene risico's + demo-video

Voor vraag 4 van de opdracht hebben we een demo gemaakt die je [hier](#) kan bekijken.

5. Ecosysteemplaatje van een IoT-omgeving



Waar zitten de grootste risico's?

1. De gateway of hub

De gateway is een centrale schakel:

- alle deviceverkeer loopt hier langs
- protocolvertaling gebeurt hier
- één compromis kan veel apparaten raken

Waarom groot risico?

Single point of failure. Wie de gateway overneemt, kan vaak meeluisteren of sturen.

2. Cloudplatform en API's

Hier zitten vaak:

- gebruikersaccounts
- apparaatdata
- configuraties
- remote control functies

Risico's

- zwakke authenticatie
- te brede API-rechten
- datalekken
- slechte segmentatie tussen klanten
- foutieve opslag van keys/tokens

3. Mobiele app of webportaal

- onveilige sessies
- slechte autorisatie
- secrets in app code
- onvoldoende MFA voor beheer

Waarom belangrijk?

Gebruikers en beheerders sturen hiermee het hele systeem aan.

4. Firmware update-keten

Een heel groot risico dat vaak vergeten wordt:

- kwaadwillende update
- niet-ondertekende firmware
- update via onbeveiligd kanaal
- supply-chain compromis

Waarom erg?

Een malafide update kan permanent controle geven over alle apparaten.

5. Netwerk en thuis/bedrijfsrouter

- slecht gescheiden netwerk
- IoT op hetzelfde netwerk als laptops/servers
- open poorten
- zwakke wifi-beveiliging

Waarom belangrijk?

Een geïnfecteerd IoT-apparaat kan als springplank dienen naar andere systemen.

6. Docker hardenen voor een productieomgeving

De belangrijkste maatregelen:

- **Draai containers niet als 'root':** Standaard draaien veel containers als de hoogste beheerder (root). Maak in je *Dockerfile* een specifieke, beperkte gebruiker aan.
- **Gebruik minimale base images:** Hoe minder software in een image, hoe minder kwetsbaarheden. Gebruik bijvoorbeeld *Alpine Linux* of *Distroless* images in plaats van een volledige Ubuntu-image.
- **Beperk resources (CPU/Memory):** Voorkom dat 1 gehackte container of een bug de hele server platlegt (een Denial of Service).
- **Scan op kwetsbaarheden:** Gebruik tools zoals *Trivy* om je images te scannen op bekende beveiligingslekken voordat je ze live zet.

Voorbeeld: Stel je draait een webshop in Docker. Als de webserver draait als 'root' en een hacker vindt een lek in je website, heeft hij direct de hoogste rechten op die container. Draait de webserver als de beperkte gebruiker 'web_user', dan zit de hacker vast in dat beperkte account.

7. Security functies: RA4M1 vs. ESP32-S2

De RA4M1 en de ESP32-S2 zijn allebei populaire microcontrollers, maar ze hebben een andere focus qua beveiliging.

RA4M1 (Renesas):

- **Secure Crypto Engine (SCE):** Een hardwarematige module die cryptografie (zoals AES-encryptie) heel snel uitvoert zonder de hoofdprocessor te belasten.
- **True Random Number Generator (TRNG):** Genereert onvoorspelbare willekeurige getallen voor het maken van veilige sleutels.
- **Wat kun je ermee?** Je kunt lokaal data veilig versleutelen, bijvoorbeeld sensordata op een apparaat opslaan zonder dat iemand het zomaar kan uitlezen.

ESP32-S2 (Espressif):

- **Secure Boot V2:** Controleert bij het opstarten of de software digitaal is ondertekend door jou. Zo niet, dan start de chip niet op.
- **Flash Encryption:** Versleutelt de opslagchip (flashgeheugen) waar je code op staat.
- **Wat kun je ermee?** Dit is perfect voor IoT-apparaten (Internet of Things). Het voorkomt dat concurrenten of hackers jouw slimme thermostaat openmaken en de code kopiëren of aanpassen.

De ESP32-S2 is specifiek ontworpen voor verbonden IoT-toepassingen waarbij het apparaat in onveilige omgevingen hangt (denk aan een slimme deurbel). Functies zoals *Secure Boot* en *Flash Encryption* zijn daar belangrijk. De RA4M1 heeft weliswaar basis-cryptografie ingebouwd, maar mist de zware, ingebouwde beveiliging tegen het stelen of manipuleren van firmware (code) die de ESP-chip wel heeft.

8. True Random Number Generator (TRNG) & goede encryptiesleutels

Een **True Random Number Generator (TRNG)** is een stukje hardware dat willekeurige getallen genereert op basis van onvoorspelbare, fysieke processen uit de echte wereld. Dit in tegenstelling tot een Pseudo-Random Number Generator (PRNG), die een wiskundige formule gebruikt en dus (met genoeg rekenkracht) te voorspellen is.

- **Hoe werkt het?** Een TRNG meet bijvoorbeeld thermische ruis in een chip, kleine variaties in voltage, of radio-achtergrondstraling.

Hoe weet je of je een goede sleutel hebt gegenereerd? Een goede sleutel heeft een hoge **entropie**. Entropie is een maat voor chaos of onvoorspelbaarheid.

1. **Lengte:** Hij moet lang genoeg zijn (bijv. 256-bit).
2. **Statistische spreiding:** De enen en nullen in de sleutel moeten volledig willekeurig verdeeld zijn. Beveiligingsexperts testen dit met wiskundige tests (zoals de *NIST test suite*) om te bewijzen dat er geen verborgen patroon in de uitvoer van de TRNG zit.

9. Hardening-maatregelen voor een netwerkprinter

Printers worden vaak vergeten bij netwerkbeveiliging en zijn daarom vaak een doelwit voor hackers om een bedrijfsnetwerk binnen te dringen.

De beste hardening-maatregelen:

1. **Wijzig standaard wachtwoorden:** Verander direct het 'admin' account van de webinterface. (Standaardwachtwoorden zoals admin/1234 of admin/password).
2. **Plaats de printer in een apart VLAN:** Zet de printer in een gesegmenteerd stuk van het netwerk. Gebruikers in het kantoor-VLAN mogen alleen printopdrachten sturen, maar de printer mag niet zomaar verbinding maken met het internet of de servers.
3. **Schakel ongebruikte protocollen uit:** Oude printers ondersteunen vaak onveilige protocollen zoals Telnet, FTP, of oude versies van SNMP (v1/v2). Zet deze uit en gebruik alleen veilige varianten (zoals HTTPS en SNMPv3).
4. **Update de firmware:** Zorg dat de printer de laatste software-updates heeft om bekende lekken te dichten.

Voorbeeld: Stel, een printer hangt direct aan het hoofdnetwerk zonder apart VLAN en heeft FTP nog aan staan. Een hacker verbindt via het gasten-wifi met de printer, krijgt toegang via FTP en gebruikt de printer vervolgens als 'springplank' om de database-server van het bedrijf aan te vallen. Met een VLAN en afgesloten poorten was dit onmogelijk geweest.

OPDRACHT 2

Opdracht 2 was het maken van een presentatie. De powerpoint kun je [hier](#) downloaden